

Workstation / Laptop	Dell G16 7630
Processor SKU	Intel Core i9-13900HX
Physical Cores	24
Logical SMT Threads	32
L2 Cache	32 MB
L3 Cache	36 MB
RAM	16 GB
RAM Speed	4800 MT/s configured
Python	3.14.3
OS	Windows 11

Task 1

Workers	Measured Time T(p)	Observed Speedup S(p)	Ideal Linear	Parallel Efficiency
1	2.0138 s	0.96×	1.00×	96.22%
2	1.1826 s	1.64×	2.00×	81.93%
4	0.8073 s	2.40×	4.00×	60.01%
8	0.6545 s	2.96×	8.00×	37.01%
12	0.6683 s	2.90×	12.00×	24.16%
16	0.8901 s	2.18×	16.00×	13.61%

```
PS C:\Users\ramaz\Downloads\labs> python task1_amdahl.py
OS Reported Logical Cores: 32
Single Process Baseline Time T(1): 1.9377s

Cores | Time(s) | Observed Speedup | Theoretical Linear
-----|-----|-----|-----
1 | 2.0138 | 0.96x | 1.00x
2 | 1.1826 | 1.64x | 2.00x
4 | 0.8073 | 2.40x | 4.00x
8 | 0.6545 | 2.96x | 8.00x
12 | 0.6683 | 2.90x | 12.00x
16 | 0.8901 | 2.18x | 16.00x
```

1

The observed speedup reaches its maximum at 8 workers (2.96×). Beyond this point, scaling flattens and then degrades: execution time increases from 0.6545 s at 8 workers to 0.6683 s at 12 workers and 0.8901 s at 16 workers. This indicates an empirical inflection point around $p = 8$. My Intel Core i9-13900HX has 24 physical cores (8 P-cores + 16 E-cores) and 32 logical threads. Therefore, the degradation occurs before all physical cores are utilized. The main causes are multiprocessing overhead, process scheduling, memory/cache contention, and the overhead of coordinating additional workers.

2

Using the empirical measurements, $T(1) = 1.9377$ s and $T(4) = 0.8073$ s.

According to Amdahl's Law:

$P = 0.7778$

Therefore, the measured parallel fraction is approximately 77.78%, while the non-parallel fraction is approximately 22.22%.

3

From Q1.2, $P = 0.7778$

The asymptotic speedup limit is approximately 4.50×.

Therefore, even if the department purchased a 128-core server, this particular workload would theoretically not achieve anywhere near 128× speedup. The approximately 22.22% non-parallel portion limits the maximum speedup to about 4.50×. Additional cores beyond the useful scaling range would mainly add overhead and contention rather than provide proportional performance improvement.

Task 2

```
PS C:\Users\ramaz\Downloads\labs> python task2_falsesharing.py
Adjacent Indices (False Sharing): 0.5453s
Padded Indices (Cache-Aligned): 0.5680s
Slowdown Factor: 0.96x
PS C:\Users\ramaz\Downloads\labs> python task2_falsesharing.py
Adjacent Indices (False Sharing): 0.6395s
Padded Indices (Cache-Aligned): 0.6811s
Slowdown Factor: 0.94x
PS C:\Users\ramaz\Downloads\labs> python task2_falsesharing.py
Adjacent Indices (False Sharing): 0.6842s
Padded Indices (Cache-Aligned): 0.7254s
Slowdown Factor: 0.94x
```

Memory Layout

	Trial 1	Trial 2	Trial 3	Median
Adjacent Indices (False Sharing Active)	0.5453 s	0.6395 s	0.6842 s	0.6395 s

Padded Indices (Cache-Line Aligned) 0.5680 s 0.6811 s 0.7254 s 0.6811 s

1

A 64-bit reference occupies 8 bytes. A cache line is 64 bytes. Therefore, 8 references can fit into one 64-byte cache line:

$$64/8=8$$

With a stride of 16 elements, the distance between two corresponding elements is:

$$16 \times 8 = 128$$

Since 128 bytes is greater than one 64-byte cache line, the padded elements are separated by at least two cache-line widths and therefore cannot reside within the same 64-byte cache line, assuming normal contiguous reference-array placement.

2

Initially, the cache line containing the two array elements may be in the Shared state when both cores have a copy. When Core 0 writes to index 0, it requires exclusive ownership of the cache line, causing other cached copies to be invalidated. The line in Core 0 transitions to Modified, while Core 1's copy becomes Invalid. When Core 1 subsequently writes to index 1, it must obtain ownership of the cache line, causing another coherence transaction and invalidating Core 0's copy. Therefore, even though the two threads modify different logical variables, they repeatedly invalidate each other's cache line. This is the mechanism behind false sharing.

3

Two industry-standard mechanisms are C++11 `alignas(64)` and Java's `@Contended`. `alignas(64)` allows a data structure to be aligned to a 64-byte boundary, while `@Contended` instructs the Java runtime to separate heavily contended fields using padding. Both approaches reduce the probability that independent frequently modified variables share the same cache line.

Task 3

```
True
PS C:\Users\ramaz\Downloads\labs> python task3_sync.py
Unsafe: Value = 2,000,000 / 2,000,000 | Time: 0.0992s
Locked: Value = 2,000,000 / 2,000,000 | Time: 0.2962s
Contention Cost Multiplier: 2.99x
```

Counter	Value	Time
Unsafe	2,000,000 / 2,000,000	0.0992 s
Locked	2,000,000 / 2,000,000	0.2962 s
Contention Cost	—	2.99×

1

Observed result: The UnsafeCounter returned exactly 2,000,000 / 2,000,000, so no lost updates were observed in this execution. Its execution time was 0.0992 s. The LockedCounter also returned 2,000,000 / 2,000,000, but required 0.2962 s. The measured contention cost multiplier was 2.99×.

In a race condition, `self.val += 1` can conceptually be decomposed into a `LOAD → ADD → STORE` sequence. If two threads read the same old value before either stores the updated value, one update can overwrite the other, producing a lost update. However, this particular Python 3.14.3 execution with the GIL enabled did not empirically show such corruption.

2

Lockless Map-Reduce result: The redesigned implementation produced exactly 2,000,000 / 2,000,000 operations in 0.0495 s. The LockedCounter required 0.2962 s.

The measured speedup is:

$$0.2962 / 0.0495 = 5.98\times$$

Therefore, the lockless implementation was approximately 5.98× faster than the LockedCounter in this experiment. Each thread accumulated its result in private local state, and the partial results were combined with a final `sum()` after all threads joined. This removed the per-operation lock acquisition and contention.

The lockless redesign removes the shared mutable counter from the hot loop. Each thread updates its own local accumulator without acquiring a lock. The partial results are written to separate array elements and reduced once after all threads finish. Therefore, synchronization is removed from the critical path, reducing lock acquisition overhead and contention. In this experiment, this resulted in a $5.98\times$ speedup compared with the `LockedCounter`.